

آموزش FPGA

با VHDL و Verilog

تهیه و تألیف: محمد مهدی قائمی

Mahdighaemi123@gmail.com

Codewithmahdi.ir

@codewithmahdi

V1.0.0

فهرست مطالب

فصل ۱: آشنایی با FPGA و معماری داخلی	۵
۱. مقدمه FPGA: چیست؟	۵
۲. مقایسه گول‌های پردازی: جایگاه FPGA کجاست؟	۶
۳. معماری داخلی: داخل این چیپ چه خبر است؟	۷
۴. منابع سخت‌افزاری آماده (Hard IP)	۱۰
۵. جریان طراحی: از ایده تا عمل	۱۰

فصل ۲: زبان VHDL؛ دستور پخت سخت افزار ۱۲

- ۱: توهم برنامه نویسی: تفاوت اجرای موازی و ترتیبی ۱۲
- ۲: مفهوم کتابخانه ها و پکیج ها (Library & Package) ۱۳
- ۳: موجودیت و درگاه ها (Entity & Port) ۱۳
- ۴: معماری: (Architecture) نقشه داخلی مدار ۱۴
- ۵: انواع سیم های داخلی (Signal & Constant) ۱۵
- ۶: پروژه نهایی فصل: طراحی مدار کلید دو پل ۱۵

فصل ۴: محاسبات ریاضی در سخت افزار؛ ساخت یک ماشین حساب هوشمند ۱۷

- ۱: چرا به کتابخانه های جدید نیاز داریم؟ (جعبه ابزار ریاضی) ۱۷
- ۲: تفاوت انواع داده ها Unsigned در مقابل Signed ۱۸
- ۳: عملیات الحاق (Concatenation) با علامت & ۱۸
- ۴: فرآیند سه مرحله ای محاسبات (Casting) ۱۹
- ۵: متغیرها (Variable)؛ یادداشت های موقتی و سریع ۲۰
- ۶: عملگرهای در دسترس در VHDL ۲۱
- ۷: مفهوم بیت نقلی و سرریز (Carry & Overflow) ۲۱
- ۸: پروژه نهایی فصل: واحد محاسبات و منطق (Mini ALU) ۲۲

فصل ۵: حافظه در سخت افزار؛ زمان بندی و ضربان قلب مدار ۲۴

- ۱: مفهوم کلاک یا ضربان قلب (Clock) ۲۴
- ۲: فلیپ فلاپ؛ کوچک ترین واحد حافظه (D-Flip-Flop) ۲۵
- ۳: بازنشانی یا شروع از نقطه صفر (Reset) ۲۵
- ۴: سیگنال های داخلی و مقداردهی (Others) ۲۶
- ۵: پروژه فصل: طراحی یک شمارنده ۴ بیتی (Binary Counter) ۲۶

فصل ۶: آلبوم قطعات دیجیتال؛ از گیت های ساده تا سازه های پیچیده ۲۹

- ۱: نیم جمع کننده (Half Adder) ۲۹

۳۰	۲: جمع‌کننده کامل (Full Adder)
۳۱	۳: فلیپ‌فلاپ نوع D (The D Flip-Flop)
۳۲	۴: ثبات جابجاگر (Shift Register)
۳۳	۵: مقایسه‌گر عددی (Digital Comparator)

فصل ۷: پروژه‌های پیشرفته؛ از مغز پردازنده تا کنترلرهای هوشمند

۳۴	۱: واحد محاسبه و منطق ۸ بیتی (Professional ALU)
۳۶	۲: مدیریت نمایشگر سون‌سگمنت (7-Segment Controller)
۳۷	۳: تولیدکننده موج PWM برای کنترل موتور
۳۸	۴: ماشین وضعیت چراغ راهنمایی (Traffic Light FSM)
۳۹	۵: حافظه رم تک پورت (Single Port RAM)

فصل ۸: زبان Verilog؛ الفبای طراحی سریع و چابک

۴۰	۱: فلسفه Verilog و تفاوت در نگاه
۴۰	۲: چهار وضعیت منطقی (Logic Levels)
۴۱	۳: روش استاندارد نوشتن اعداد
۴۱	۴: اسکلت‌بندی ماژول (Module)
۴۲	۵: انواع داده‌ها؛ سیم‌ها در مقابل ظرف‌ها (Wire vs Reg)
۴۲	۶: مقداردهی مستقیم (Assign)
۴۳	۷: بلوک‌های همیشه فعال (Always Block)
۴۳	۸: تفاوت دو علامت مساوی = و <=

فصل ۹: عملگرها و جادوی محاسبات در Verilog

۴۴	۱: عملگرهای ریاضی (Arithmetic Operators)
۴۵	۲: عملگرهای منطقی و بیتی (Logical vs Bitwise)
۴۵	۳: عملگر الحاق (Concatenation) و تکرار
۴۵	۴: پروژه اول؛ نیم‌جمع‌کننده (Half Adder)

۵: پروژه دوم؛ جمع‌کننده کامل (Full Adder) ۴۶

۶: پروژه سوم؛ جمع‌کننده ۴ بیتی حرفه‌ای ۴۶

فصل ۱۰: پروژه‌های پیشرفته وریلوگ؛ مدیریت زمان و حافظه ۴۸

۱: حافظه و ضربان قلب مدار (The Clock) ۴۹

۲: پروژه اول؛ شمارنده ۴ بیتی هوشمند (Binary Counter) ۵۰

۳: پروژه دوم؛ کنترل‌کننده نور و موتور (PWM Controller) ۵۱

۴: پروژه سوم؛ ماشین وضعیت چراغ راهنمایی (Traffic Light FSM) ۵۲

۵: پروژه چهارم؛ حافظه رم تک‌پورت (Single Port RAM) ۵۳

فصل ۱: آشنایی با FPGA و معماری داخلی

۱. مقدمه FPGA: چیست؟

برای شروع بیایید یک ذهنیت مهم را اصلاح کنیم. وقتی به یک CPU یا میکروکنترلر نگاه می‌کنید، شما با یک سخت‌افزار ثابت روبرو هستید که فقط دستورات نرم‌افزاری شما را اجرا می‌کند. اما FPGA داستان متفاوتی دارد.

FPGA مخفف **Field Programmable Gate Array** است. این یعنی «آرایه‌ای از گیت‌های منطقی که توسط شما در میدان عمل برنامه‌ریزی می‌شوند»¹. تصور کنید FPGA یک زمین بازی پر از قطعات لگو است. شما می‌توانید با این قطعات یک روز ماشین بسازید و روز دیگر هواپیما. در دنیای الکترونیک یعنی شما سخت‌افزار را توصیف می‌کنید و FPGA تبدیل به آن سخت‌افزار می‌شود.

در واقع FPGA نماینده رویکرد پردازش مکانی است. در پردازنده‌ها کارها نوبت‌بندی شده و در طول زمان اجرا می‌شوند، اما در FPGA عملیات در پهنه فضا و روی سطح تراشه پخش شده و همزمان اجرا می‌شوند

۲. مقایسه غول‌های پردازشی: جایگاه FPGA کجاست؟

به عنوان یک مهندس باید بدانید چه زمانی از کدام قطعه استفاده کنید. بیایید چهار بازیگر اصلی دنیای دیجیتال را مقایسه کنیم:

پردازنده مرکزی (CPU)

- ماهیت: مغز متفکر کامپیوترها.
- عملکرد: عالی برای تصمیم‌گیری‌های پیچیده و اجرای سیستم‌عامل.
- نقطه ضعف: کارها را سریال انجام می‌دهد (یکی پس از دیگری). حتی سریع‌ترین CPUها هم باید دستورات را از حافظه بخوانند و اجرا کنند.³

پردازنده گرافیکی (GPU)

- ماهیت: متخصص کارهای موازی.
- عملکرد: دارای هزاران هسته کوچک است که برای کارهای گرافیکی و هوش مصنوعی عالی هستند.
- نقطه ضعف: همچنان وابسته به دستورالعمل است و تاخیر (Latency) بالایی دارد، یعنی برای کارهای کنترلی خیلی سریع مناسب نیست.⁴⁴⁴⁴

مدار مجتمع خاص منظوره (ASIC)

- ماهیت: سخت‌افزار اختصاصی و ثابت.
- عملکرد: سریع‌ترین و کم‌مصرف‌ترین چیپ دنیاست چون دقیقاً برای یک هدف خاص ساخته می‌شود.
- نقطه ضعف: هزینه طراحی آن میلیون‌ها دلار است و اگر اشتباه کنید راه برگشتی وجود ندارد

تراشه FPGA

- ماهیت: سخت افزار انعطاف پذیر.
- عملکرد: سرعت آن بسیار بالاتر از CPU است و قابلیت انجام کارهای موازی را دارد.
- نقطه قوت: مثل ASIC سریع است اما مثل نرم افزار انعطاف پذیر است. اگر در طراحی اشتباه کردید، فقط کد را اصلاح و دوباره آپلود می کنید.^۶

جدول خلاصه مقایسه

ویژگی	FPGA	CPU	ASIC
نوع پردازش	موازی (سخت افزاری)	سریال (نرم افزاری)	سخت افزار ثابت
انعطاف پذیری	بالا (تغییر سخت افزار)	بالا (تغییر برنامه)	صفر (غیر قابل تغییر)
هزینه اولیه	متوسط	کم	بسیار زیاد
کاربرد	مخابرات، رادار، پردازش سیگنال	کنترل و سیستم عامل	تولید انبوه (موبایل)

۳. معماری داخلی: داخل این چیپ چه خبر است؟

وقتی می گوئیم FPGA را برنامه ریزی می کنیم، دقیقاً چه چیزی را تغییر می دهیم؟ معماری داخلی FPGA های مدرن (مثل سری ۷ زایلینکس) از سه بخش اصلی تشکیل شده است: ۷:

الف) جدول‌های جستجو یا LUT -> قلب تپنده

در FPGA ما گیت‌های AND و OR فیزیکی نداریم. به جای آن از حافظه‌های کوچکی به نام LUT استفاده می‌کنیم. هر تابع منطقی در واقع یک جدول صحت است. FPGA ورودی‌ها را به عنوان آدرس حافظه می‌گیرد و خروجی را از جدول می‌خواند. پس ما با تغییر محتوای این حافظه‌ها، رفتار مدار را تغییر می‌دهیم.

مثال عملی: ساخت یک گیت AND با LUT بیایید ببینیم چطور یک LUT دو ورودی

می‌تواند نقش یک گیت AND را بازی کند.

۱. می‌دانیم که جدول صحت گیت AND به شکل زیر است:

- ورودی ۰۰ -> خروجی ۰
- ورودی ۰۱ -> خروجی ۰
- ورودی ۱۰ -> خروجی ۰
- ورودی ۱۱ -> خروجی ۱

۲. حالا این جدول صحت را دقیقاً درون حافظه LUT کپی می‌کنیم. این حافظه ۴ خانه دارد (چون ۲ ورودی، ۴ حالت ایجاد می‌کند):

- در آدرس (۰۰) حافظه، مقدار ۰ را ذخیره می‌کنیم.
- در آدرس (۰۱) حافظه، مقدار ۰ را ذخیره می‌کنیم.
- در آدرس (۱۰) حافظه، مقدار ۰ را ذخیره می‌کنیم.
- در آدرس (۱۱) حافظه، مقدار ۱ را ذخیره می‌کنیم.

نتیجه: حالا اگر به ورودی‌های این LUT سیگنال‌های ۱ و ۱ بدهیم، LUT به آدرس ۱۱

مراجعه کرده و مقدار ذخیره شده یعنی «۱» را خروجی می‌دهد

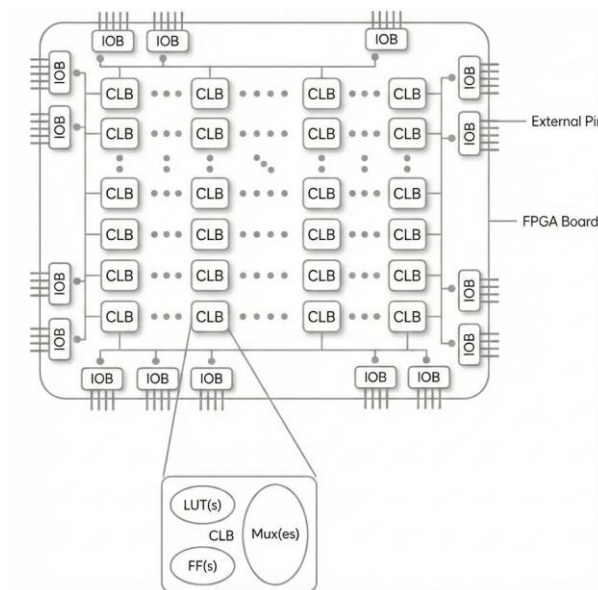
قدرت FPGA: اگر بخواهیم همین مدار را به گیت OR تبدیل کنیم، نیازی به تغییر سیم‌کشی نیست؛ فقط کافی است محتویات حافظه را از 0001 به 0111 تغییر دهیم. به همین سادگی سخت‌افزار تغییر کرد!

ب) فلیپ‌فلاپ‌ها و اسلایس‌ها

برای ساخت مدارهای ترتیبی و حافظه‌دار، کنار هر LUT یک فلیپ‌فلاپ قرار دارد. مجموعه‌ای از چند LUT و فلیپ‌فلاپ در بسته‌هایی به نام Slice و CLB گروه‌بندی می‌شوند که بلوک‌های سازنده اصلی چیپ هستند.

ج) سیم‌کشی یا Routing

شاید تعجب کنید اما ۹۰ درصد مساحت چیپ را سیم‌ها و سوئیچ‌ها اشغال کرده‌اند. این شبکه عظیم وظیفه دارد بلوک‌های منطقی را به هم وصل کند. کیفیت این سیم‌کشی تعیین می‌کند که مدار شما چقدر می‌تواند سریع کار کند.



۴. منابع سخت افزاری آماده (Hard IP)

مهندسان متوجه شدند که ساختن برخی مدارها با استفاده از LUT های معمولی خیلی پرهزینه و کند است. بنابراین برخی بلوک های پرکاربرد را به صورت آماده و بهینه شده در دل FPGA قرار دادند:

- **بلاک های محاسباتی (DSP):** مدارهای ضرب کننده و جمع کننده آماده که برای کارهای ریاضی سنگین مثل فیلترهای دیجیتال عالی هستند.
- **حافظه های داخلی (Block RAM):** حافظه های رم داخلی که برای ذخیره داده ها استفاده می شوند. ویژگی مهم آنها این است که همزمان می توانند از دو پورت مختلف خوانده و نوشته شوند.
- **مدیریت کلاک (MMCM):** بلوک هایی که می توانند فرکانس کلاک ورودی را تغییر دهند، مثلاً کلاک ۵۰ مگاهرتز را به ۱۰۰ مگاهرتز تبدیل کنند.¹³

۵. جریان طراحی: از ایده تا عمل

برای اینکه یک طرح را روی FPGA پیاده کنید، باید مراحل زیر را طی کنید. این پروسه شبیه کامپایل کردن برنامه نیست، بلکه شبیه ساختن یک ساختمان است:

۱. **نوشتن کد (Design Entry):** رفتار مدار را با زبان هایی مثل VHDL یا Verilog توصیف می کنید.
۲. **سنتز (Synthesis):** نرم افزار کد شما را می خواند و آن را به قطعات اولیه FPGA مثل LUT و فلیپ فلاپ ترجمه می کند.
۳. **پیاده سازی (Implementation):** سنگین ترین مرحله است. نرم افزار تصمیم می گیرد هر تکه از مدار را در کجای چیپ قرار دهد (Place) و چگونه آنها را به هم سیم کشی کند. (Route)

۴. تولید فایل بیت (Bitstream) در نهایت یک فایل باینری تولید می‌شود که با کابل روی FPGA بارگذاری شده و سخت‌افزار شما شکل می‌گیرد.

نکته پایانی:

یادگیری FPGA یعنی یادگیری مدیریت فیزیک سیلیکون. برخلاف برنامه‌نویسی معمولی که نگران نوبت اجرا هستید، اینجا باید نگران زمان‌بندی و همزمانی باشید. این دنیایی است که در آن «زمان» و «مکان» معنی جدیدی پیدا می‌کنند.

فصل ۲: زبان VHDL ؛ دستور پختِ سخت افزار

در این فصل یاد می گیریم که چگونه با نوشتن کد، درون یک تراشه را سیم کشی کنیم. تصور کنید که قصد دارید یک مدار را روی یک برد برد طراحی کنید VHDL. دقیقاً همین کار را به صورت متنی برای شما انجام می دهد.

۱: توهم برنامه نویسی: تفاوت اجرای موازی و ترتیبی

بزرگ ترین نکته ای که باید در نظر بگیرید این است که VHDL یک زبان برنامه نویسی نیست، بلکه یک زبان توصیف سخت افزار است. در زبان هایی مثل پایتون، پردازنده خط اول را اجرا می کند و تا تمام نشود به سراغ خط دوم نمی رود. این یعنی اجرای ترتیبی. اما در سخت افزار، همه چیز همزمان اتفاق می افتد. یک اتوبان بزرگ را تصور کنید که تمام ماشین ها در باندهای مختلف با هم حرکت می کنند. اگر شما ده تا لامپ را به ده تا کلید وصل کنید، به محض اینکه کلیدها را بزنید، همه لامپ ها با هم روشن می شوند. در VHDL هم وقتی چند دستور را می نویسید، تراشه برای هر کدام یک مدار جداگانه می سازد که همه آنها با هم کار می کنند.

مثال کد برای درک بهتر:

```
1. -- In VHDL, these two lines don't wait for each other
2. -- They represent two separate wires working at the same time
3. Fan <= Switch_1;
4. Light <= Switch_2;
```

در این مثال، روشن شدن فن هیچ تاخیری در روشن شدن چراغ ایجاد نمی‌کند چون هر دو مسیر به صورت فیزیکی و همزمان در تراشه ساخته شده‌اند.

۲: مفهوم کتابخانه‌ها و پکیج‌ها (Library & Package)

قبل از شروع نوشتن هر کدی، باید ابزارهای خود را انتخاب کنیم. زبان VHDL در حالت پایه بسیار محدود است. برای اینکه بتوانیم از انواع سیم‌های پیشرفته و استانداردهای جهانی استفاده کنیم، باید از کتابخانه IEEE استفاده کنیم.

- **Library:** مثل یک سوله بزرگ است که ابزارهای مختلفی در آن قرار دارد. با نوشتن `library IEEE;` ما در این سوله را باز می‌کنیم.

- **Package:** درون این سوله، جعبه‌ابزارهای مختلفی وجود دارد. معروف‌ترین آن‌ها `std_logic_1164` است. این جعبه‌ابزار به ما اجازه می‌دهد از نوع داده‌ای به نام `std_logic` استفاده کنیم که استاندارد طلایی در طراحی دیجیتال است.

نحوه نوشتن:

```
1. -- Opening the library
2. library IEEE;
3. -- Using everything inside the standard logic package
4. use IEEE.STD_LOGIC_1164.ALL;
```

۳: موجودیت و درگاه‌ها (Entity & Port)

هر مدار یک پوسته خارجی دارد. در VHDL این پوسته را با کلمه **Entity** تعریف می‌کنیم. در این بخش، ما پایه‌های ورودی و خروجی مدار را مشخص می‌کنیم.

مفهوم Port: پورت‌ها در واقع همان سیم‌هایی هستند که از جعبه مدار ما بیرون آمده‌اند. هر پورت دو ویژگی اصلی دارد:

۱. جهت: که می‌تواند ورودی یعنی **in** یا خروجی یعنی **out** باشد
۲. نوع: که مشخص می‌کند چه چیزی از این سیم عبور می‌کند.

مفهوم std_logic: این یکی از مهم‌ترین مفاهیم است. به جای استفاده از **bit** که فقط صفر و یک می‌فهمد، از **std_logic** استفاده می‌کنیم. این نوع داده بسیار هوشمند است و می‌تواند حالت‌های مختلفی مثل سیم قطع شده یا ولتاژ نامشخص را هم نمایش دهد. در اکثر پروژه‌های واقعی، همیشه از این نوع استفاده می‌شود.

مثال بخش: Entity

```
1. -- Defining the outer shell of our device
2. entity MyFirstChip is
3.     port (
4.         -- A and B are input wires
5.         A : in std_logic;
6.         B : in std_logic;
7.         -- Q is the output wire
8.         Q : out std_logic
9.     );
10. end MyFirstChip;
```

۴: معماری (Architecture): نقشه داخلی مدار

حالا که پوسته‌ی مدار و پایه‌ها را ساختیم، باید بگوییم داخل این جعبه چه خبر است. این کار را در بخش **Architecture** انجام می‌دهیم. اینجا جایی است که سیم‌کشی‌ها و منطق مدار را می‌نویسیم.

در این بخش از علامت **=>** برای مقداردهی استفاده می‌کنیم. این علامت یعنی مقدار سمت راست را بر روی سیم سمت چپ قرار بده.

مثال معماری برای یک مدار ساده:

```
1. -- Defining what happens inside MyFirstChip
2. architecture InternalLayout of MyFirstChip is
3. begin
4.     -- Connecting Q to the result of A AND B
5.     Q <= A and B;
6. end InternalLayout;
```

۵: انواع سیم‌های داخلی (Signal & Constant)

گاهی نیاز داریم دو قطعه را در داخل مدار به هم وصل کنیم، بدون اینکه سیمی از جعبه خارج شود. در این صورت از **Signal** استفاده می‌کنیم. سیگنال‌ها مثل سیم‌های رابط روی برد مورد هستند. همچنین اگر بخواهیم یک مقدار ثابت مثل عدد ۱ را در جایی همیشه ثابت نگه داریم، از **Constant** استفاده می‌کنیم.

مثال استفاده:

```
1. architecture Wiring logic of MyDevice is
2.     -- Defining an internal wire that is not a Port
3.     signal internal_wire : std_logic;
4.     -- Defining a constant value
5.     constant activated : std_logic := '1';
6. begin
7.     -- Logic goes here
8.     internal_wire <= Input_1 and Input_2;
9.     Output_1 <= internal_wire or activated;
10. end Wiring;
```

۶: پروژه نهایی فصل: طراحی مدار کلید دو پل

بیا باید تمام قطعات پازل را کنار هم بگذاریم. می‌خواهیم مداری طراحی کنیم که مثل کلیدهای راهرو عمل کند. یعنی دو کلید داریم که اگر وضعیت آن‌ها متفاوت باشد، لامپ روشن شود. این کار با گیت منطقی **XOR** انجام می‌شود.

کد کامل و استاندارد:

```
1. -- Step 1: Loading the necessary tools
2. library IEEE;
3. use IEEE.STD_LOGIC_1164.ALL;
4.
5. -- Step 2: Defining the inputs and outputs
6. entity StaircaseController is
7.     port (
8.         Switch_A : in std_logic; -- Input from the first switch
9.         Switch_B : in std_logic; -- Input from the second switch
10.        Lamp      : out std_logic -- Output to the lamp
11.    );
12. end StaircaseController;
13.
14. -- Step 3: Defining the logic inside
15. architecture Behavioral of StaircaseController is
16. begin
17.     -- The lamp is ON if switches are in different positions
18.     -- This is the definition of XOR gate
19.     Lamp <= Switch_A xor Switch_B;
20. end Behavioral;
```

تحلیل نهایی: در این کد، ما ابتدا کتابخانه را معرفی کردیم تا سیستم بفهمد std_logic چیست. سپس در بخش موجودیت، سه پایه برای مدار تعریف کردیم. در نهایت در بخش معماری، با یک خط کد ساده، منطق کلید دو پل را پیاده کردیم. این کد حالا آماده است تا به سخت افزار واقعی تبدیل شود.

فصل ۴: محاسبات ریاضی در سخت افزار؛ ساخت یک ماشین حساب هوشمند

در دنیای دیجیتال، همه چیز در نهایت به صفر و یک ختم می شود. اما وقتی می خواهیم بگوییم «دو عدد را با هم جمع کن»، باید راهی باشد که به زبان VHDL بفهمانیم این دسته های سیم فقط سیگنال نیستند، بلکه نمایانگر یک عدد با ارزش ریاضی هستند.

۱: چرا به کتابخانه های جدید نیاز داریم؟ (جعبه ابزار ریاضی)

همان طور که قبلاً یاد گرفتیم، زبان VHDL به صورت پیش فرض بسیار خام است. کتابخانه IEEE.STD_LOGIC_1164 که همیشه در ابتدای کد می آوریم، فقط به ما اجازه می دهد از سیم های استاندارد استفاده کنیم. این سیم ها فقط وضعیت هایی مثل صفر یا یک بودن را می فهمند و هیچ درکی از عملیات ریاضی ندارند.

برای اینکه بتوانیم عملگرهایی مثل جمع و تفریق را به کار ببریم، باید از کتابخانه تخصصی NUMERIC_STD استفاده کنیم. این کتابخانه استاندارد طلائی در صنعت است و به ما اجازه می دهد محاسبات را به صورت دقیق و بهینه انجام دهیم.

نحوه فراخوانی کتابخانه‌ها در ابتدای کد:

```
1. -- This is the base library for standard wires
2. library IEEE;
3. use IEEE.STD_LOGIC_1164.ALL;
4.
5. -- This library enables math operations like addition and subtraction
6. use IEEE.NUMERIC_STD.ALL;
```

۲: تفاوت انواع داده‌ها Unsigned در مقابل Signed

در ریاضیات دیجیتال، ما باید مشخص کنیم که اعداد ما چگونه تفسیر شوند. در VHDL دو نوع اصلی برای کار با اعداد وجود دارد:

۱. نوع **Unsigned** این نوع برای اعدادی استفاده می‌شود که همیشه مثبت هستند. مثلاً اگر ۴ سیم داشته باشیم، می‌توانیم اعداد ۰ تا ۱۵ را نشان دهیم. ۲. نوع **Signed** این نوع برای اعدادی است که می‌توانند مثبت یا منفی باشند. در اینجا بیت سمت چپ به عنوان علامت عدد در نظر گرفته می‌شود.

مثال تعریف در کد:

```
1. -- A bundle of 4 wires that represents a positive number from 0 to 15
2. signal positive_number : unsigned(3 downto 0);
3.
4. -- A bundle of 4 wires that can represent numbers from -8 to +7
5. signal signed_number : signed(3 downto 0);
```

۳: عملیات الحاق (Concatenation) با علامت &

قبل از انجام جمع، باید با یک علامت جادویی آشنا شویم. علامت & در VHDL برای چسباندن دو سیم یا دو دسته سیم به هم استفاده می‌شود. چرا این کار مهم است؟ وقتی می‌خواهید دو عدد ۴ بیتی را با هم جمع کنید، حاصل ممکن است ۵ بیت شود. برای اینکه

فضای کافی داشته باشیم، یک بیت صفر به ابتدای عدد می‌چسبانیم تا ظرف محاسباتی ما بزرگتر شود.

مثال الحاق:

```
1. -- Adding a '0' to the beginning of a 4-bit wire to make it 5 bits
2. extended_wire <= '0' & original_4bit_wire;
```

۴: فرآیند سه مرحله‌ای محاسبات (Casting)

این بخش حساس‌ترین قسمت کار است. ورودی‌ها و خروجی‌های مدار ما همیشه از نوع `std_logic_vector` هستند که یعنی فقط «دسته سیم» هستند. برای انجام ریاضیات، باید این مسیر را طی کنیم:

۱. مرحله تبدیل به عدد: ابتدا دسته‌های سیم ورودی را به نوع `unsigned` یا `signed` تبدیل می‌کنیم.

۲. مرحله محاسبه: عملیات ریاضی مثل جمع یا تفریق را روی اعداد انجام می‌دهیم.

۳. مرحله تبدیل به سیم: حاصل را دوباره به `std_logic_vector` تبدیل می‌کنیم تا بتوانیم آن را به خارج از تراشه بفرستیم.

مثال کد این فرآیند:

```
1. -- Step 1: Wire to Number
2. num_A <= unsigned(wire_A);
3.
4. -- Step 2: Math operation
5. num_Result <= num_A + num_B;
6.
7. -- Step 3: Number to Wire
8. wire_Result <= std_logic_vector(num_Result);
```

۵: متغیرها (Variable)؛ یادداشت‌های موقتی و سریع

تا اینجا یاد گرفتیم که سیگنال‌ها مثل سیم هستند. اما در دنیای VHDL ابزار دیگری به نام **Variable** داریم که مثل یک کاغذچکنویس برای محاسبات سریع عمل می‌کند.

تفاوت‌های اصلی متغیر با سیگنال در این موارد است:

۱. **مکان حضور**: متغیرها فقط و فقط داخل یک **Process** تعریف می‌شوند و خارج از آن هیچ هویتی ندارند.

۲. **سرعت آپدیت**: برخلاف سیگنال که تا پایان یافتن فرآیند صبر می‌کند تا مقدارش عوض شود، متغیر همان لحظه و در همان خط، مقدار جدید را به خود می‌گیرد.

۳. **علامت مقداردهی**: برای سیگنال از `<=` استفاده می‌کنیم اما برای متغیر از علامت `=` استفاده می‌شود.

چرا از متغیر استفاده می‌کنیم؟ وقتی می‌خواهید یک محاسبه چند مرحله‌ای انجام دهید، اگر از سیگنال استفاده کنید، به دلیل تاخیر در آپدیت، ممکن است در محاسبات لحظه‌ای دچار خطا شوید. اما متغیرها چون بلافاصله تغییر می‌کنند، برای محاسبات میانی و لحظه‌ای در سخت‌افزار بسیار بهینه و سریع هستند.

مثال کد برای درک تفاوت: در این مثال می‌بینید که چطور یک مقدار را موقتاً در متغیر نگه می‌داریم تا نتیجه نهایی را بسازیم.

```
1. process(A, B)
2.     -- Variables are defined before the 'begin' of a process
3.     variable intermediate_sum : unsigned(3 downto 0);
4. begin
5.     -- 1. Temporary calculation using ':= '
6.     intermediate_sum := unsigned(A) + 1;
7.
8.     -- 2. Using the updated variable immediately in the next line
9.     -- If this was a signal, it would still have its OLD value here!
10.    Result <= std_logic_vector(intermediate_sum + unsigned(B));
11.
12. end process;
```

یک نکته طلایی برای شما: همیشه یادتان باشد که متغیرها مثل روح هستند؛ درون تراشه سیم‌کشی فیزیکی دائمی ندارند و فقط برای کمک به منطق محاسباتی شما در لحظه سنتز استفاده می‌شوند. پس برای جابجایی داده بین بخش‌های مختلف مدار، هنوز هم پادشاه مطلق، همان Signal است.

۶: عملگرهای در دسترس در VHDL

وقتی کتابخانه ریاضی را اضافه می‌کنید، می‌توانید از تمام این عملگرها استفاده کنید:

- عملگرهای محاسباتی: + برای جمع، - برای تفریق و * برای ضرب.
- عملگرهای مقایسه‌ای: = برای چک کردن برابری، > برای بزرگتر بودن و < برای کوچکتر بودن.

نکته مهم اینجاست که شما فقط با نوشتن یک علامت + در واقع دارید به نرم‌افزار می‌گویید که مجموعه‌ای پیچیده از صدها گیت منطقی را در دل تراشه برای شما سیم‌کشی کند.

۷: مفهوم بیت نقلی و سرریز (Carry & Overflow)

در سخت‌افزار، ما محدودیت فیزیکی داریم. اگر بالاترین عدد ممکن را با یک عدد دیگر جمع کنیم، ظرف ما پر می‌شود و یک بیت به بیرون پرتاب می‌شود. به این بیت، بیت نقلی (Carry) می‌گوییم. مدیریت این بیت در طراحی ماشین‌حساب‌ها حیاتی است، چون به ما می‌گوید که آیا نتیجه محاسبه درست است یا از حد مجاز فراتر رفته است.

۸: پروژه نهایی فصل: واحد محاسبات و منطق (Mini ALU)

در این پروژه، ما یک مدار کامل طراحی می‌کنیم که دو عدد ۴ بیتی را می‌گیرد. این مدار یک کلید انتخاب به نام Control دارد. اگر این کلید صفر باشد، اعداد را جمع می‌کند و اگر یک باشد، آن‌ها را از هم کم می‌کند. همچنین یک چراغ برای نمایش بیت نقلی (Carry) داریم.

کد کامل:

```

1. -- First, we bring the essential math tools
2. library IEEE;
3. use IEEE.STD_LOGIC_1164.ALL;
4. use IEEE.NUMERIC_STD.ALL;
5.
6. -- Defining the inputs and outputs
7. entity MiniALU is
8.     port (
9.         Data_A : in  std_logic_vector(3 downto 0); -- First 4-bit input
10.        Data_B : in  std_logic_vector(3 downto 0); -- Second 4-bit input
11.        Control : in  std_logic;                  -- 0 for Add, 1 for Sub
12.        Result  : out std_logic_vector(3 downto 0); -- 4-bit main result
13.        Carry   : out std_logic                    -- Overflow flag
14.    );
15. end MiniALU;
16.
17. -- The logic architecture of our calculator
18. architecture Behavioral of MiniALU is
19. begin
20.     -- This process handles the math logic
21.     process(Data_A, Data_B, Control)
22.         -- We use 5-bit variables to handle the extra carry bit
23.         variable temp_A : unsigned(4 downto 0);
24.         variable temp_B : unsigned(4 downto 0);
25.         variable temp_Res : unsigned(4 downto 0);
26.     begin
27.         -- Step 1: Extend to 5 bits and convert to unsigned
28.         -- We add a '0' at the front to avoid overflow during math
29.         temp_A := unsigned('0' & Data_A);
30.         temp_B := unsigned('0' & Data_B);
31.
32.         -- Step 2: Choose the operation
33.         if Control = '0' then
34.             -- Addition mode
35.             temp_Res := temp_A + temp_B;
36.         else
37.             -- Subtraction mode
38.             temp_Res := temp_A - temp_B;
39.         end if;
40.
41.         -- Step 3: Assign results back to the output ports
42.         -- We take the lower 4 bits for the sum result
43.         Result <= std_logic_vector(temp_Res(3 downto 0));

```

```
44.  
45.         -- We take the 5th bit (index 4) as the carry flag  
46.         Carry <= temp_Res(4);  
47.  
48.     end process;  
49. end Behavioral;
```

تحلیل جامع کد پروژه

در این کد، ما از تمام مفاهیم فصل استفاده کردیم :

۱. در بخش **Entity**، پورت‌های ورودی و خروجی را به صورت دسته سیم ۴ تایی تعریف کردیم .

۲. در بخش **Architecture** ، یک **process** ایجاد کردیم چون نیاز به استفاده از دستور **if** داشتیم

۳. از **Variable** در داخل فرآیند استفاده کردیم، چون متغیرها در **VHDL** برای محاسبات میانی و لحظه‌ای بسیار سریع‌تر و بهتر از سیگنال‌ها عمل می‌کنند.

۴. با استفاده از الحاق یا همان **&** اعداد را به ۵ بیت گسترش دادیم تا اگر حاصل جمع از ۱۵ بیشتر شد، اطلاعات ما از بین نرود و در بیت پنجم ذخیره شود .

۵. در نهایت، با استفاده از برش زدن (**temp_Res(3 downto 0)**، فقط بخش اصلی عدد را به خروجی فرستادیم و بیت اضافی را به خروجی **Carry** متصل کردیم.

فصل ۵: حافظه در سخت افزار؛

زمان بندی و ضربان قلب مدار

در این فصل می فهمیم که چطور می توانیم اطلاعات را ذخیره کنیم. تفاوت اصلی این فصل با فصل های قبلی در وجود مفهومی به نام زمان است. مدارهای قبلی ما حافظه نداشتند، اما مدارهای این فصل مثل یک دفترچه یادداشت هستند که می توانند وضعیت قبلی خود را حفظ کنند.

۱: مفهوم کلاک یا ضربان قلب (Clock)

در سخت افزار دیجیتال، برای اینکه تمام بخش های یک تراشه با نظم و هماهنگی پیش بروند، به یک سیگنال هماهنگ کننده نیاز داریم که به آن کلاک می گوئیم. کلاک یک سیگنال است که مدام بین صفر و یک نوسان می کند.

تصور کنید یک گروه موسیقی در حال اجرا است. رهبر ارکستر با حرکت چوب خود، زمان دقیق نواختن هر نت را مشخص می کند. کلاک دقیقاً همین نقش را در تراشه ایفا می کند. تمام اتفاقات مهم مدار، دقیقاً در لحظه ای که کلاک از صفر به یک تغییر وضعیت می دهد، رخ می دهند. به این لحظه خاص، لبه صعودی یا **Rising Edge** می گوئیم.

۲: فلیپ‌فلاپ؛ کوچک‌ترین واحد حافظه (D-Flip-Flop)

فلیپ‌فلاپ قطعه‌ای است که می‌تواند یک بیت داده یعنی صفر یا یک را در خود ذخیره کند.

معروف‌ترین نوع آن D-Flip-Flop نام دارد. روش کار آن به این صورت است:

- این قطعه یک ورودی داده به نام D و یک خروجی به نام Q دارد.
- تا زمانی که کلاک تغییر نکند، خروجی Q مقدار قبلی خود را حفظ می‌کند و به ورودی توجهی ندارد.
- به محض اینکه لبه صعودی کلاک فرا برسد، مقدار فعلی ورودی D خوانده شده و در خروجی Q ذخیره می‌شود.

توضیح دستور: rising_edge این یک تابع جادویی در VHDL است که بررسی می‌کند آیا سیگنال کلاک از مقدار صفر به یک تغییر کرده است یا خیر. این دستور تضمین می‌کند که مدار ما فقط در لحظات خاصی تغییر وضعیت دهد.

۳: بازنشانی یا شروع از نقطه صفر (Reset)

هر مدار حافظه‌داری نیاز دارد که در لحظه شروع به کار یا در مواقع اضطراری، به یک حالت امن و شناخته شده برگردد. به این کار ریست کردن می‌گوییم. ما دو نوع اصلی ریست داریم:

1. **ریست همزمان (Synchronous):** فقط وقتی کلاک بزند، مدار ریست می‌شود
2. **ریست غیرهمزمان (Asynchronous):** به محض اینکه دکمه ریست زده شود، مدار بدون منتظر ماندن برای کلاک، فوراً صفر می‌شود. در طراحی‌های صنعتی معمولاً از این روش استفاده می‌شود چون امنیت بالاتری دارد.

مثال کد یک حافظه ۱ بیتی با ریست غیرهمزمان:

```
1. -- This process monitors both Clock and Reset signals
2. process(Clk, Rst)
3. begin
4.     -- First priority: check if reset button is pressed
5.     if Rst = '1' then
6.         Q <= '0'; -- Clear the memory immediately
7.     -- Second priority: check for the heartbeat of the circuit
8.     elsif rising_edge(Clk) then
9.         Q <= D; -- Store the input data on the clock edge
10.    end if;
11. end process;
```

۴: سیگنال‌های داخلی و مقداردهی (Others)

در مدارهای حافظه‌دار، ما اغلب نیاز داریم که یک دسته سیم بزرگ را به صورت همزمان صفر کنیم. به جای اینکه تک‌تک بیت‌ها را بنویسیم، از عبارت `others => '0'` استفاده می‌کنیم. این عبارت به VHDL می‌گوید که فرقی نمی‌کند این دسته سیم چقدر بزرگ است، تمام بیت‌های آن را برابر صفر قرار بده.

همچنین به یاد داشته باشید که ما نمی‌توانیم مقداری را از یک پورت خروجی بخوانیم. پس همیشه یک **Signal** داخلی تعریف می‌کنیم تا محاسبات را روی آن انجام دهیم و در نهایت مقدار آن را به پورت خروجی بفرستیم.

۵: پروژه فصل: طراحی یک شمارنده ۴ بیتی (Binary Counter)

در این پروژه می‌خواهیم مداری طراحی کنیم که با هر ضربان کلاک، یک واحد به عدد قبلی خود اضافه کند. این مدار دارای یک ورودی کلاک، یک ورودی ریست و یک خروجی ۴ بیتی است که عدد فعلی را نشان می‌دهد. این شمارنده از صفر تا پانزده می‌شمارد و سپس دوباره به صفر برمی‌گردد.

کد کامل و استاندارد پروژه:

```

1. -- Standard library for wire types
2. library IEEE;
3. use IEEE.STD_LOGIC_1164.ALL;
4. -- Standard library for arithmetic operations
5. use IEEE.NUMERIC_STD.ALL;
6.
7. -- Entity defining the ports of our counter
8. entity TopCounter is
9.     port (
10.         Clk      : in  std_logic;           -- System clock input
11.         Rst       : in  std_logic;           -- System reset input
12.         Digit     : out std_logic_vector(3 downto 0) -- 4-bit counter
13.     );
14. end TopCounter;
15.
16. -- Architecture describing how the counter works
17. architecture Behavioral of TopCounter is
18.     -- Internal signal to keep track of the count
19.     -- We use unsigned type because we want to do math
20.     signal count_internal : unsigned(3 downto 0) := "0000";
21. begin
22.     -- Sequential process for counting logic
23.     process(Clk, Rst)
24.     begin
25.         -- If Reset is active, set count to zero immediately
26.         if Rst = '1' then
27.             count_internal <= (others => '0');
28.
29.             -- On every rising edge of the clock, increment the value
30.             elsif rising_edge(Clk) then
31.                 -- Adding 1 to the current internal value
32.                 count_internal <= count_internal + 1;
33.             end if;
34.         end process;
35.
36.         -- Send the internal signal to the output port
37.         -- Converting unsigned type back to standard logic vector
38.         Digit <= std_logic_vector(count_internal);
39.
40.     end Behavioral;

```

تحلیل دقیق عملکرد پروژه

۱. بخش کتابخانه‌ها: ما از NUMERIC_STD استفاده کردیم چون می‌خواستیم عملیات جمع را روی شمارنده انجام دهیم
۲. سیگنال داخلی: یک سیگنال به نام count_internal تعریف کردیم. این سیگنال در واقع همان حافظه مدار ماست که مقدار فعلی را در خود نگه می‌دارد.
۳. لیست حساسیت: در ابتدای process نام Clk و Rst را نوشتیم. این یعنی هر بار که کلاک بزند یا دکمه ریست فشار داده شود، این بخش از کد بیدار شده و اجرا می‌شود.
۴. منطق ریست: ریست را در ابتدای شرط قرار دادیم تا بالاترین اولویت را داشته باشد. یعنی حتی اگر کلاک بزند اما دست ما روی دکمه ریست باشد، مدار باید صفر بماند.
۵. بخش جمع: در هر لبه صعودی، مقدار فعلی با ۱ جمع شده و دوباره در خود سیگنال ذخیره می‌شود. چون این سیگنال ۴ بیتی است، بعد از رسیدن به عدد ۱۵ (۱۱۱۱)، به صورت خودکار سرریز کرده و صفر می‌شود.

فصل ۶: آلبوم قطعات دیجیتال؛ از گیت‌های ساده تا سازه‌های پیچیده

در این فصل یاد می‌گیریم که چگونه قطعات معروف مهندسی دیجیتال را با VHDL توصیف کنیم. این فصل برای شما مثل یک مرجع خواهد بود که هر زمان نیاز به یک قطعه خاص داشتید، به آن مراجعه کنید.

۱: نیم‌جمع‌کننده (Half Adder)

ساده‌ترین واحد محاسباتی در سخت‌افزار، نیم‌جمع‌کننده است. این قطعه دو ورودی یک‌بیتی را می‌گیرد و حاصل جمع و بیت نقلی (Carry) را تولید می‌کند. چون این مدار ورودی برای بیت نقلی قبلی ندارد، به آن «نیم» جمع‌کننده می‌گوییم.

منطق مدار:

- مجموع (Sum): حاصل XOR دو ورودی است.
- خروجی نقلی (Carry): حاصل AND دو ورودی است.

مثال کد:

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity HalfAdder is
5.     port (
6.         A      : in  std_logic; -- First input bit
7.         B      : in  std_logic; -- Second input bit
8.         Sum     : out std_logic; -- Addition result
9.         Carry   : out std_logic -- Carry out bit
10.    );
11. end HalfAdder;
12.
13. architecture Behavioral of HalfAdder is
14. begin
15.     -- Simple combinational logic without process
16.     Sum  <= A xor B;
17.     Carry <= A and B;
18. end Behavioral;

```

۲: جمع کننده کامل (Full Adder)

وقتی بخواهیم چندین بیت را با هم جمع کنیم، هر مرحله باید بتواند بیت نقلی مرحله قبل را هم دریافت کند. جمع کننده کامل سه ورودی دارد: دو بیت اصلی و یک بیت نقلی از مرحله قبل (Cin).

مثال کد:

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity FullAdder is
5.     port (
6.         A      : in  std_logic; -- First bit
7.         B      : in  std_logic; -- Second bit
8.         Cin     : in  std_logic; -- Carry in from previous stage
9.         S      : out std_logic; -- Sum output
10.        Cout   : out std_logic -- Carry out to next stage
11.    );
12. end FullAdder;
13.
14. architecture Logic of FullAdder is
15. begin
16.     -- Formula for full adder logic

```

```
17.     S    <= A xor B xor Cin;
18.     Cout <= (A and B) or (Cin and (A xor B));
19. end Logic;
```

۳: فلیپ‌فلاپ نوع D (The D Flip-Flop)

همان‌طور که در فصل قبل و در متن کتاب خواندیم، فلیپ‌فلاپ کوچکترین واحد حافظه در FPGA است. این قطعه داده را در لبه صعودی کلاک ذخیره می‌کند و تا لبه بعدی ثابت نگه می‌دارد. در واقع این قطعه مرز بین دنیای ترکیبی و دنیای ترتیبی است.

مثال کد با ریست غیرهمزمان:

```
1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity D_FlipFlop is
5.     port (
6.         Clk    : in  std_logic; -- Clock signal (the heartbeat)
7.         Reset   : in  std_logic; -- Asynchronous reset
8.         D       : in  std_logic; -- Data input
9.         Q       : out std_logic  -- Data output (stored value)
10.    );
11. end D_FlipFlop;
12.
13. architecture Storage of D_FlipFlop is
14. begin
15.     process(Clk, Reset)
16.     begin
17.         if Reset = '1' then
18.             -- Reset has the highest priority
19.             Q <= '0';
20.         elsif rising_edge(Clk) then
21.             -- Store data only on the rising edge of clock
22.             Q <= D;
23.         end if;
24.     end process;
25. end Storage;
```

۴: ثبات جابجاگر (Shift Register)

حالا بیا یک سازه کمی پیچیده‌تر بسازیم. ثبات جابجاگر مجموعه‌ای از فلیپ‌فلاپ‌ها است که داده‌ها را در هر ضربان کلاک، یک خانه به جلو هل می‌دهند. این قطعه برای تبدیل داده‌های سریال (که پشت سر هم می‌آیند) به موازی (که با هم خارج می‌شوند) بسیار پرکاربرد است.

مثال کد یک جابجاگر ۴ بیتی:

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity ShiftRegister4 is
5.     port (
6.         Clk      : in  std_logic; -- Clock
7.         Si       : in  std_logic; -- Serial Input (the data coming in)
8.         Parallel_Out : out std_logic_vector(3 downto 0) -- 4-bit output
9.     );
10. end ShiftRegister4;
11.
12. architecture Shifting of ShiftRegister4 is
13.     -- Internal signal to act as the 4-bit storage chain
14.     signal internal_reg : std_logic_vector(3 downto 0) := "0000";
15. begin
16.     process(Clk)
17.     begin
18.         if rising_edge(Clk) then
19.             -- Moving data: bit 2 goes to 3, bit 1 to 2, bit 0 to 1, and
20.             new data to 0
21.             -- This is called bit shifting
22.             internal_reg <= internal_reg(2 downto 0) & Si;
23.         end if;
24.     end process;
25.     -- Drive the internal storage to the output pins
26.     Parallel_Out <= internal_reg;
27. end Shifting;

```

۵: مقایسه‌گر عددی (Digital Comparator)

گاهی نیاز داریم بدانیم آیا دو عدد با هم برابر هستند یا یکی از دیگری بزرگتر است. در فصل ۴ یاد گرفتیم که کتابخانه NUMERIC_STD اجازه استفاده از علامت‌های مقایسه‌ای را به ما می‌دهد.

مثال کد مقایسه‌گر ۴ بیتی:

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. entity MagnitudeComparator is
6.     port (
7.         A      : in  std_logic_vector(3 downto 0);
8.         B      : in  std_logic_vector(3 downto 0);
9.         A_Greater : out std_logic;
10.        Equal    : out std_logic
11.    );
12. end MagnitudeComparator;
13.
14. architecture Compare logic of MagnitudeComparator is
15. begin
16.     process(A, B)
17.     begin
18.         -- Using unsigned casting to compare as numbers
19.         if unsigned(A) > unsigned(B) then
20.             A_Greater <= '1';
21.             Equal <= '0';
22.         elsif unsigned(A) = unsigned(B) then
23.             A_Greater <= '0';
24.             Equal <= '1';
25.         else
26.             A_Greater <= '0';
27.             Equal <= '0';
28.         end if;
29.     end process;
30. end architecture;
```

فصل ۷: پروژه‌های پیشرفته؛ از مغز پردازنده تا کنترلرهای هوشمند

در این فصل ۵ پروژه کاربردی و کامل را با هم بررسی می‌کنیم. هر پروژه شامل کد استاندارد و تحلیل دقیق است تا درک عمیقی از نحوه اتصال قطعات پیدا کنید.

۱: واحد محاسبه و منطق ۸ بیتی (Professional ALU)

واحد ALU قلب تپنده هر پردازنده است که وظیفه انجام محاسبات ریاضی و منطقی را بر عهده دارد. این قطعه بر اساس یک کد انتخاب یا همان Selection مشخص می‌کند که چه عملیاتی روی ورودی‌ها انجام شود. ما در اینجا از کتابخانه تخصصی ریاضی استفاده می‌کنیم تا محاسبات دقیق انجام دهیم.

ویژگی‌های پروژه:

- انجام جمع و تفریق روی اعداد ۸ بیتی.
- انجام عملیات منطقی مثل AND و OR و XOR.
- مدیریت بیت نقلی یا همان Carry برای تشخیص سرریز محاسبات.

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL; -- Tool for math operations
4.
5. entity AdvancedALU is
6.     port (
7.         A      : in  std_logic_vector(7 downto 0); -- 8-bit input A
8.         B      : in  std_logic_vector(7 downto 0); -- 8-bit input B
9.         Sel     : in  std_logic_vector(2 downto 0); -- 3-bit mode select
10.        Result  : out std_logic_vector(7 downto 0); -- 8-bit result
11.        Carry   : out std_logic;                    -- Overflow flag
12.        Zero    : out std_logic                     -- Status flag for zero
13.    );
14. end AdvancedALU;
15.
16. architecture Behavioral of AdvancedALU is
17. begin
18.     process(A, B, Sel)
19.         -- Using a 9-bit variable to capture the carry bit
20.         variable temp_res : unsigned(8 downto 0);
21.     begin
22.         case Sel is
23.             when "000" => -- Addition mode
24.                 temp_res := unsigned('0' & A) + unsigned('0' & B);
25.             when "001" => -- Subtraction mode
26.                 temp_res := unsigned('0' & A) - unsigned('0' & B);
27.             when "010" => -- Logical AND
28.                 temp_res := unsigned('0' & (A and B));
29.             when "011" => -- Logical OR
30.                 temp_res := unsigned('0' & (A or B));
31.             when "100" => -- Logical XOR
32.                 temp_res := unsigned('0' & (A xor B));
33.             when others =>
34.                 temp_res := (others => '0'); -- Default case
35.         end case;
36.
37.         -- Mapping variable back to ports
38.         Result <= std_logic_vector(temp_res(7 downto 0));
39.         Carry <= temp_res(8); -- The 9th bit is our carry
40.
41.         -- Setting the Zero flag
42.         if temp_res(7 downto 0) = 0 then
43.             Zero <= '1';
44.         else
45.             Zero <= '0';
46.         end if;
47.     end process;
48. end Behavioral;

```

۲: مدیریت نمایشگر سون سگمنت (7-Segment Controller)

برای نمایش اعداد در دنیای واقعی به یک مبدل نیاز داریم. این مدار یک عدد ۴ بیتی باینری را می‌گیرد و آن را به کد مخصوص روشن کردن ۷ عدد LED تبدیل می‌کند. هر عدد در واقع یک جدول صحت است که درون حافظه‌های LUT تراشه ذخیره می‌شود. ما از نوع داده هوشمند std_logic استفاده می‌کنیم تا تمام حالت‌ها را پوشش دهیم.

ویژگی‌های پروژه:

- تبدیل مستقیم اعداد ۰ تا ۹ به کد سگمنت.
- مدیریت حالت‌های تعریف نشده برای امنیت مدار.

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3.
4. entity SevenSegDecoder is
5.     port (
6.         Digit      : in  std_logic_vector(3 downto 0); -- 4-bit binary input
7.         Segments    : out std_logic_vector(6 downto 0) -- a,b,c,d,e,f,g bits
8.     );
9. end SevenSegDecoder;
10.
11. architecture Logic of SevenSegDecoder is
12. begin
13.     process(Digit)
14.     begin
15.         -- Mapping binary to 7-segment hex code
16.         case Digit is
17.             when "0000" => Segments <= "1111110"; -- Displays 0
18.             when "0001" => Segments <= "0110000"; -- Displays 1
19.             when "0010" => Segments <= "1101101"; -- Displays 2
20.             when "0011" => Segments <= "1111001"; -- Displays 3
21.             when "0100" => Segments <= "0110011"; -- Displays 4
22.             when "0101" => Segments <= "1011011"; -- Displays 5
23.             when "0110" => Segments <= "1011111"; -- Displays 6
24.             when "0111" => Segments <= "1110000"; -- Displays 7
25.             when "1000" => Segments <= "1111111"; -- Displays 8
26.             when "1001" => Segments <= "1111011"; -- Displays 9
27.             when others => Segments <= "0000001"; -- Error dash (-)
28.         end case;
29.     end process;
30. end Logic;

```

۳: تولیدکننده موج PWM برای کنترل موتور

این پروژه برای کنترل سرعت موتورهای یا نور LED استفاده می‌شود. ما از یک کلاک به عنوان ضربان قلب مدار استفاده می‌کنیم. با مقایسه یک شمارنده داخلی با مقدار ورودی می‌توانیم زمان روشن بودن خروجی را تنظیم کنیم. این مدار در واقع ترکیبی از یک حافظه و یک واحد محاسباتی است.

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. entity PWM_Controller is
6.     port (
7.         Clk      : in  std_logic;           -- Global clock
8.         Target    : in  std_logic_vector(7 downto 0); -- Speed or brightness
9.         PWM_Pin   : out std_logic           -- Physical output pin
10.    );
11. end PWM_Controller;
12.
13. architecture Behavioral of PWM_Controller is
14.     signal timer : unsigned(7 downto 0) := (others => '0'); --
15. begin
16.     process(Clk)
17.     begin
18.         if rising_edge(Clk) then -- Triggered on clock s heartbeat
19.             timer <= timer + 1; -- Increment counter
20.
21.             -- Decision making logic
22.             if timer < unsigned(Target) then
23.                 PWM_Pin <= '1';
24.             else
25.                 PWM_Pin <= '0';
26.             end if;
27.         end if;
28.     end process;
29. end Behavioral;

```

۴: ماشین وضعیت چراغ راهنمایی (Traffic Light FSM)

این پروژه مغز متفکر سیستم را نشان می‌دهد. ما از فلیپ‌فلاپ‌ها برای ذخیره وضعیت فعلی استفاده می‌کنیم. سیستم بر اساس زمان‌بندی کلاک بین وضعیت‌های قرمز و زرد و سبز جابجا می‌شود. این بهترین مثال برای درک مدیریت زمان و مکان در FPGA است.

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. entity TrafficSystem is
6.     port (
7.         Clk : in std_logic;
8.         Rst : in std_logic; -- Reset to initial state
9.         Lights: out std_logic_vector(2 downto 0) -- Red, Yellow, Green
10.    );
11. end TrafficSystem;
12.
13. architecture SmartLogic of TrafficSystem is
14.     -- Custom type for finite states
15.     type state_type is (Red_State, Yellow_State, Green_State);
16.     signal current_s : state_type;
17.     signal count      : unsigned(23 downto 0) := (others => '0');
18. begin
19.     process(Clk, Rst)
20.     begin
21.         if Rst = '1' then -- Asynchronous reset
22.             current_s <= Red_State;
23.             count <= (others => '0');
24.         elsif rising_edge(Clk) then
25.             count <= count + 1;
26.
27.             case current_s is
28.                 when Red_State =>
29.                     Lights <= "100"; -- Red light on
30.                     if count > 10000000 then -- Simple delay
31.                         current_s <= Green_State;
32.                         count <= (others => '0');
33.                     end if;
34.                 when Green_State =>
35.                     Lights <= "001"; -- Green light on
36.                     if count > 10000000 then
37.                         current_s <= Yellow_State;
38.                         count <= (others => '0');
39.                     end if;
40.                 when Yellow_State =>
41.                     Lights <= "010"; -- Yellow light on

```

```

42.             if count > 2000000 then
43.                 current_s <= Red_State;
44.                 count <= (others => '0');
45.             end if;
46.         end case;
47.     end if;
48. end process;
49. end SmartLogic;

```

۵: حافظه رم تک پورت (Single Port RAM)

در FPGA منابع آماده‌ای به نام Block RAM وجود دارد که برای ذخیره داده‌های انبوه عالی هستند. این مدار به شما اجازه می‌دهد در یک آدرس مشخص داده‌ای را بنویسید یا از آن بخوانید. این پروژه از ترکیب آرایه‌ها و سیگنال‌های داخلی برای شبیه‌سازی یک حافظه واقعی استفاده می‌کند.

```

1. library IEEE;
2. use IEEE.STD_LOGIC_1164.ALL;
3. use IEEE.NUMERIC_STD.ALL;
4.
5. entity InternalMemory is
6.     port (
7.         Clk : in std_logic;
8.         WE : in std_logic; -- Write Enable signal
9.         Addr : in std_logic_vector(3 downto 0); -- 16 memory locations
10.        Data_In : in std_logic_vector(7 downto 0); -- 8-bit data input
11.        Data_Out : out std_logic_vector(7 downto 0) -- 8-bit data output
12.    );
13. end InternalMemory;
14.
15. architecture Storage of InternalMemory is
16.    -- Defining an array of 16 memory slots
17.    type ram_type is array (0 to 15) of std_logic_vector(7 downto 0);
18.    signal my_ram : ram_type := (others => (others => '0'));
19. begin
20.     process(Clk)
21.     begin
22.         if rising_edge(Clk) then
23.             if WE = '1' then
24.                 -- Writing data to specific address
25.                 my_ram(to_integer(unsigned(Addr))) <= Data_In;
26.             end if;
27.             -- Reading data from specific address
28.             Data_Out <= my_ram(to_integer(unsigned(Addr)));
29.         end if;
30.     end process;
31. end Storage;

```

فصل ۸: زبان Verilog؛ الفبای طراحی

سریع و چابک

در این فصل، تمرکز اصلی ما روی درک ساختار و قواعد نگارشی (سینتکس) است. وریلوگ به ما کمک می‌کند تا با کلماتی فشرده‌تر و کدی کوتاه‌تر، همان سخت‌افزاری را توصیف کنیم که قبلاً یاد گرفتیم.

۱: فلسفه Verilog و تفاوت در نگاه

زبان Verilog بر پایه رویکرد «پردازش مکانی» ساخته شده است. در برنامه‌نویسی مثل پایتون یا C، شما نگران این هستید که کدام دستور «اول» اجرا شود. اما در وریلوگ، ما نگران این هستیم که قطعات در کدام «بخش از فضا» روی سطح تراشه چیده شوند. بر خلاف پردازنده‌های معمولی که کارها را نوبت‌بندی کرده و پشت سر هم انجام می‌دهند، در وریلوگ تمام بخش‌های مدار همزمان با هم کار می‌کنند. مثال برای درک تفاوت: تصور کنید دو مسیر مجزا برای دو لامپ دارید. در دنیای وریلوگ، این دو خط کد همزمان و بدون معطلی اجرا می‌شوند.

```
1. // Both lines are active at the exact same time
2. assign Green_LED = Switch_1; // Path one
3. assign Red_LED = Switch_2; // Path two
```

۲: چهار وضعیت منطقی (Logic Levels)

در دنیای دیجیتال واقعی، سیم‌ها فقط صفر و یک نیستند. برای اینکه یک مدار واقعی به درستی شبیه‌سازی شود، وریلوگ چهار حالت فیزیکی را برای هر سیم می‌شناسد:

- 0 مقدار صفر منطقی یا ولتاژ پایین (Low).
 - 1 مقدار یک منطقی یا ولتاژ بالا (High).
 - X (Unknown) وضعیت نامشخص. زمانی که مدار هنوز پایدار نشده یا دو سیگنال مخالف (۰ و ۱) همزمان به یک نقطه وصل شده‌اند.
 - Z (High Impedance) وضعیت امپدانس بالا. یعنی سیم از نظر فیزیکی به هیچ ولتاژی وصل نیست و به نوعی «روی هوا» است.
- مثال کاربردی: یک سیستم را در نظر بگیرید که یک پورت خروجی دارد اما فعلاً داده‌ای برای ارسال ندارد؛ در این حالت پورت را در وضعیت Z قرار می‌دهیم تا در مدار اختلال ایجاد نکند.

۳: روش استاندارد نوشتن اعداد

- در وریلوگ شما نمی‌توانید فقط بنویسید عدد ۸؛ چون تراشه باید بداند این ۸ قرار است چند رشته سیم (بیت) را اشغال کند. فرمت استاندارد نوشتن اعداد به این صورت است:
- تعداد بیت: مشخص می‌کند که این عدد در حافظه چقدر جا می‌گیرد.
 - مبنا: شامل b باینری، d دهدهی و h شانزدهدهی
- مثال‌های کد:

```
1. 4'b1010 // 4 bits, binary value 1010 (equals 10)
2. 8'hA5   // 8 bits, hexadecimal value A5
3. 1'b1    // 1 bit, binary value 1
```

۴: اسکلت‌بندی ماژول (Module)

در وریلوگ، تمام توصیف مدار شما درون یک بلوک واحد به نام **Module** قرار می‌گیرد. این ماژول مثل یک جعبه است که ورودی‌ها به آن وارد و خروجی‌ها از آن خارج می‌شوند. دیگر خبری از جداسازی پیچیده مثل Entity و Architecture در VHDL نیست.

مثال ساختار یک ماژول ساده:

```
1. // Defining the module name and ports
2. module MyBasicCircuit (
3.     input wire A,    // Input pin
4.     output wire Q    // Output pin
5. );
6.
7.     // Connecting the pins
8.     assign Q = A;
9.
10. endmodule // End of the module
```

۵: انواع داده‌ها؛ سیم‌ها در مقابل ظرف‌ها (Wire vs Reg)

درک این بخش، کلید اصلی شما برای کدنویسی درست است. ما دو مدل اصلی برای تعریف مسیرهای داده داریم:

۱ **Wire** سیم : دقیقاً مثل یک سیم مسی است. سیم نمی‌تواند مقداری را در خود ذخیره کند و فقط سیگنال را عبور می‌دهد. اگر منبع ورودی قطع شود، سیم دیگر مقداری ندارد.

۲ **Reg** ثبات : این کلمه از Register می‌آید. برخلاف سیم، reg مثل یک حافظه کوچک عمل کرده و می‌تواند یک مقدار را در خود نگه دارد. هر متغیری که قرار است داخل یک بلوک محاسباتی مثل always مقدار بگیرد، باید از این نوع باشد.

مثال کد:

```
1. wire external_connection; // Just a wire path
2. reg internal_storage;     // A variable that can hold data
```

۶: مقداردهی مستقیم (Assign)

ساده‌ترین راه برای وصل کردن دو قطعه در مدارهای ترکیبی (بدون حافظه)، استفاده از دستور assign است. این دستور یک اتصال دائمی ایجاد می‌کند؛ یعنی خروجی همیشه و در هر لحظه پیرو ورودی است.

مثال کد:

```
1. // Permanent connection logic
2. assign Light_ON = (Switch_A & Switch_B); // AND gate logic
```

۷: بلوک‌های همیشه فعال (Always Block)

وقتی مدار ما نیاز به تصمیم‌گیری دارد مثل استفاده از دستورات `if` یا `case`، باید از بلوک `always` استفاده کنیم. این بلوک به یک «لیست حساسیت» گوش می‌دهد؛ یعنی می‌گوید: هر وقت این اتفاق افتاد، کدهای داخل من را اجرا کن

مثال سینتکس:

```
1. // This block runs every time A or B changes
2. always @(A, B) begin
3.     // Logical decisions go here
4.     if (A == 1'b1)
5.         Q = B;
6.     else
7.         Q = 1'b0;
8. end
```

نکته: بلوک `always` در Verilog معادل `process` در VHDL است. هر دو وظیفه دارند یک بخش از کد را به تغییرات سیگنال‌های خاصی حساس کنند و اجازه دهند از دستورات تصمیم‌گیری (IF/CASE) استفاده کنیم.

۸: تفاوت دو علامت مساوی = و <=

وریلوگ دو نوع مساوی دارد که معنای کاملاً متفاوتی در سخت‌افزار دارند:

۱. **مساوی مسدودکننده (=)**: مثل زبان‌های برنامه‌نویسی معمولی (C) است. دستورات خط به خط اجرا می‌شوند. از این علامت در مدارهای ساده استفاده می‌کنیم.

۲. **مساوی غیرمسدودکننده (<=)**: این علامت مخصوص دنیای موازی سخت‌افزار است. تمام خطوطی که با این علامت نوشته شوند، در یک لحظه و به صورت همزمان اجرا می‌شوند. این علامت پادشاه مدارهای کلاک‌دار و حافظه‌دار است.

مثال مقایسه‌ای:

```
1. // Swapping values in a single clock pulse
2. A <= B;
3. B <= A; // This happens simultaneously, values are swapped!
```

فصل ۹: عملگرها و جادوی محاسبات

Verilog در

در این بخش تمرکز ما بر روی ابزارهایی است که به ما اجازه می‌دهند با دسته‌های سیم مثل اعداد واقعی رفتار کنیم.

۱: عملگرهای ریاضی (Arithmetic Operators)

یکی از بزرگ‌ترین مزیت‌های وریلوگ این است که برای انجام یک جمع ساده نیاز به نوشتن توابع پیچیده ندارید. وقتی شما از علامت + استفاده می‌کنید نرم‌افزار سنتز به صورت خودکار مجموعه‌ای از گیت‌های منطقی و LUTها را در دل تراشه برای شما سیم‌کشی می‌کند.

- + جمع: برای اضافه کردن دو مقدار به هم.
- - تفریق: برای کم کردن مقادیر.
- * ضرب: برای ضرب کردن که البته باید حواستان به تعداد بیت‌های خروجی باشد.

مثال کد:

```
1. // Simple Math Operations
2. assign Sum = A + B;           // Hardware creates an Adder
3. assign Diff = A - B;         // Hardware creates a Subtractor
4. assign Product = A * B;      // Hardware creates a Multiplier
```

نکته مهم: همیشه به یاد داشته باشید که در سخت‌افزار فضای ما محدود است. اگر حاصل جمع دو عدد از تعداد بیت‌های پورت خروجی بیشتر شود با پدیده سرریز یا **Overflow** مواجه می‌شویم.

۲: عملگرهای منطقی و بیتی (Logical vs Bitwise)

در وریلوگ دو نوع عملگر برای کارهای منطقی داریم که نباید آن‌ها را با هم اشتباه بگیرید. عملگرهای بیتی نظیر به نظیر روی بیت‌ها اثر می‌گذارند اما عملگرهای منطقی کل دسته سیم را یکجا می‌بینند.

- عملگرهای بیتی (~, ^, |, &) این‌ها روی بیت‌های متناظر اثر می‌گذارند.
- عملگرهای منطقی (!, ||, &&) این‌ها معمولاً در دستورات **if** برای بررسی شروط استفاده می‌شوند.

مثال کد:

```
1. // Difference between Bitwise and Logical
2. wire [1:0] A = 2'b10;
3. wire [1:0] B = 2'b01;
4.
5. assign Y1 = A & B; // Result is 2'b00 (Bit by bit AND)
6. assign Y2 = A && B; // Result is 1'b1 (True AND True is True)
```

۳: عملگر الحاق (Concatenation) و تکرار

در VHDL از علامت & برای چسباندن سیم‌ها استفاده می‌کردیم. در وریلوگ این کار با علامت {} انجام می‌شود. همچنین می‌توانیم با علامت {n{}} یک سیم را چند بار تکرار کنیم.

مثال سینتکس:

```
1. // Concatenation: Joining bits together
2. assign Extended_Bus = {1'b0, A}; // Adds a zero to the start
3.
4. // Replication: Repeating a bit
5. assign Bus_8bit = {8{1'b1}}; // Creates "11111111"
```

این کار دقیقاً زمانی کاربرد دارد که بخواهیم فضای کافی برای بیت نقلی یا همان Carry در محاسبات فراهم کنیم.

۴: پروژه اول؛ نیم جمع کننده (Half Adder)

بیا باید آموخته‌هایمان را در یک مدار واقعی پیاده کنیم. نیم‌جمع‌کننده دو ورودی یک‌بیتی می‌گیرد و حاصل جمع (Sum) و بیت نقلی (Carry) را تولید می‌کند.

منطق مدار:

- مجموع حاصل XOR دو ورودی است.
- بیت نقلی حاصل AND دو ورودی است.

مثال کد:

```
1. module HalfAdder (
2.     input A, B,          // Input bits
3.     output S, C          // Sum and Carry outputs
4. );
5.     // Using simple continuous assignment
6.     assign S = A ^ B; // XOR operator for Sum
7.     assign C = A & B; // AND operator for Carry
8. endmodule
```

۵: پروژه دوم؛ جمع کننده کامل (Full Adder)

این قطعه علاوه بر دو بیت اصلی یک بیت نقلی از مرحله قبل (Cin) را هم دریافت می‌کند. در اینجا می‌بینید که چقدر سینتکس وریلوگ فشرده و شیرین است.

مثال کد:

```
1. module FullAdder (
2.     input A, B, Cin, // Three input bits
3.     output S, Cout   // Sum and Carry-out
4. );
5.     // Describing the logic in one step
6.     assign S = A ^ B ^ Cin; // Logic for Sum
7.     assign Cout = (A & B) | (Cin & (A ^ B)); // Logic for Carry Out
8. endmodule
```

۶: پروژه سوم؛ جمع کننده ۴ بیتی حرفه‌ای

حالا بیایید قدرت واقعی وریلوگ را ببینید. به جای اینکه برای هر بیت جداگانه کد بنویسیم از عملگر + استفاده می‌کنیم تا یک جمع کننده ۴ بیتی بسازیم. این همان منطقی است که در پروژه‌های پیشرفته استفاده می‌شود.

مثال کد:

```
1. module Adder4Bit (
```

```
2.    input [3:0] Data_A, // 4-bit input A
3.    input [3:0] Data_B, // 4-bit input B
4.    output [3:0] Sum,    // 4-bit result
5.    output Carry        // Overflow bit
6. );
7.    // Internal wire to hold the 5-bit result (Sum + Carry)
8.    wire [4:0] temp_res;
9.
10.   // Performing the addition
11.   assign temp_res = Data_A + Data_B;
12.
13.   // Splitting the result back to ports
14.   assign Sum = temp_res[3:0]; // Taking the first 4 bits
15.   assign Carry = temp_res[4]; // The 5th bit is our Carry
16. endmodule
```

فصل ۱۰: پروژه‌های پیشرفته وریلوگ؛ مدیریت زمان و حافظه

در این فصل از ترکیب بلوک‌های محاسباتی و حافظه برای ساخت سیستم‌های پیچیده استفاده می‌کنیم. تمرکز ما بر روی استفاده از بلوک‌های همیشه فعال کلاک‌دار و مدیریت وضعیت‌های مختلف مدار است.

۱: حافظه و ضربان قلب مدار (The Clock)

در دنیای دیجیتال کلاک مانند ضربان قلب عمل می‌کند که تمام بخش‌های تراشه را با هم هماهنگ نگه می‌دارد. برای اینکه مداری داشته باشیم که چیزی را ذخیره کند باید از لبه صعودی کلاک یا همان **posedge** استفاده کنیم. کوچک‌ترین واحدی که این کار را برای ما انجام می‌دهد فلیپ‌فلاپ نام دارد. مثال کد فلیپ‌فلاپ **D** با ریست غیرهمزمان: در طراحی‌های صنعتی همیشه از ریست غیرهمزمان استفاده می‌شود چون امنیت بالاتری دارد و مدار را فوراً به حالت امن بازمی‌گرداند.

```
1. module D_FlipFlop (
2.     input clk,          // System clock
3.     input rst,          // Asynchronous reset
4.     input d,            // Data input
5.     output reg q        // Stored output
6. );
7.
8.     // This block triggers on clock pulse or reset signal
9.     always @(posedge clk or posedge rst) begin
10.         if (rst) begin
11.             // Clear memory immediately on reset
12.             q <= 1'b0;
13.         end
14.         else begin
15.             // Store data on the heartbeat of the circuit
16.             q <= d;
17.         end
18.     end
19.
20. endmodule
```

۲: پروژه اول؛ شمارنده ۴ بیتی هوشمند (Binary Counter)

شمارنده یکی از پرکاربردترین مدارها در مهندسی کامپیوتر است که با هر ضربان کلاک یک واحد به مقدار قبلی خود اضافه می‌کند. این مدار از صفر تا پانزده می‌شمارد و سپس به صورت خودکار دوباره به صفر برمی‌گردد.

کد پروژه شمارنده:

```
1. module BinaryCounter (  
2.     input clk,  
3.     input rst,  
4.     output reg [3:0] count // 4-bit storage for the number  
5. );  
6.  
7.     always @(posedge clk or posedge rst) begin  
8.         if (rst) begin  
9.             // Set counter to zero  
10.            count <= 4'b0000;  
11.        end  
12.        else begin  
13.            // Add one to the current value on every heartbeat  
14.            count <= count + 1'b1;  
15.        end  
16.    end  
17.  
18. endmodule
```

۳. پروژه دوم؛ کنترل‌کننده نور و موتور (PWM Controller)

تکنیک PWM روشی است که در آن با قطع و وصل کردن سریع جریان برق ولتاژ خروجی را برای کنترل سرعت موتور یا نور LED مدیریت می‌کنیم. ما یک مقدار هدف داریم و آن را با یک شمارنده داخلی مقایسه می‌کنیم تا زمان روشن بودن خروجی را تنظیم کنیم.

کد پروژه: PWM

```
1. module PWM_Controller (  
2.     input clk,  
3.     input [7:0] target, // Brightness or speed level from 0 to 255  
4.     output reg pwm_out // Physical output pin  
5. );  
6.  
7.     reg [7:0] timer = 8'd0; // Internal counter for timing  
8.  
9.     always @(posedge clk) begin  
10.         // The counter runs constantly in a loop  
11.         timer <= timer + 1'b1;  
12.  
13.         // Decision making: is it time to turn off the signal?  
14.         if (timer < target) begin  
15.             pwm_out <= 1'b1; // Output is high  
16.         end  
17.         else begin  
18.             pwm_out <= 1'b0; // Output is low  
19.         end  
20.     end  
21.  
22. endmodule
```

۴: پروژه سوم؛ ماشین وضعیت چراغ راهنمایی (Traffic Light FSM)

ماشین وضعیت یا FSM مغز متفکر سیستم‌های دیجیتال است که بر اساس زمان‌بندی بین وضعیت‌های مختلف جابجا می‌شود. در این مثال چراغ راهنمایی بین سه وضعیت قرمز و زرد و سبز تغییر می‌کند. این بهترین مثال برای درک مدیریت زمان در تراشه است.

```

1. module TrafficSystem (
2.     input clk,
3.     input rst,
4.     output reg [2:0] lights // Red, Yellow, Green
5. );
6.
7.     // Defining states using local parameters for better readability
8.     localparam RED = 2'b00, GREEN = 2'b01, YELLOW = 2'b10;
9.     reg [1:0] current_state;
10.    reg [23:0] timer; // Large timer for human-visible delay
11.
12.    always @(posedge clk or posedge rst) begin
13.        if (rst) begin
14.            current_state <= RED;
15.            timer <= 24'd0;
16.        end
17.        else begin
18.            timer <= timer + 1'b1;
19.            case (current_state)
20.                RED: begin
21.                    lights <= 3'b100; // Only red is on
22.                    if (timer > 24'd10000000) begin
23.                        current_state <= GREEN;
24.                        timer <= 24'd0;
25.                    end
26.                end
27.                GREEN: begin
28.                    lights <= 3'b001; // Only green is on
29.                    if (timer > 24'd10000000) begin
30.                        current_state <= YELLOW;
31.                        timer <= 24'd0;
32.                    end
33.                end
34.                YELLOW: begin
35.                    lights <= 3'b010; // Only yellow is on
36.                    if (timer > 24'd2000000) begin
37.                        current_state <= RED;
38.                        timer <= 24'd0;
39.                    end
40.                end
41.            endcase
42.        end
43.    end
44.
45. endmodule

```

۵: پروژه چهارم؛ حافظه رم تک‌پورت (Single Port RAM)

در اکثر پروژه‌های بزرگ نیاز داریم داده‌های حجیم را در بخشی از تراشه ذخیره کنیم. حافظه رم به ما اجازه می‌دهد در یک آدرس مشخص داده‌ای را بنویسیم یا از آن آدرس مقداری را بخوانیم. در وریلوگ این کار با استفاده از آرایه‌ها انجام می‌شود.

کد پروژه حافظه رم:

```
1. module SinglePortRAM (
2.     input clk,
3.     input we,           // Write Enable signal
4.     input [3:0] addr,   // Address for 16 memory locations
5.     input [7:0] data_in, // 8-bit data to be stored
6.     output reg [7:0] data_out // 8-bit retrieved data
7. );
8.
9.     // Defining an array of 16 memory slots
10.    reg [7:0] my_ram [15:0];
11.
12.    always @(posedge clk) begin
13.        if (we) begin
14.            // Writing data to a specific slot
15.            my_ram[addr] <= data_in;
16.        end
17.        // Reading data from a specific slot
18.        data_out <= my_ram[addr];
19.    end
20.
21. endmodule
```